

Functional Programming

Lecture 3 Higher-order functions

Jesper Bengtson

Last week

We introduced some key core concepts of Functional Programming

- Expression evaluations
- Function Definitions
- Lists
- User-defined types

Questions?

This week

This week

Higher-order functions

This week

Higher-order functions

- Functions take functions as arguments
- Highly useful for writing short and concise programs
- Replace loops in many cases
- Once you get to know them, programs become concise and easy to read
- Done poorly, programs become obtuse and very hard to read

Everything we do from now on depends on this lecture and the one prior. Come back to them often. Study them often. Do not wait until the exam to catch up on things you might have missed.

But first some recapitulation from last week

Anonymous functions

Simple function expressions

```
fun r -> System.Math.PI * r * r
```

returns a function that has type
float -> float

Currying

Recall

A **function with multiple arguments** is actually a function with a single argument that **returns another function**

```
fun x y -> x + y
```

This is a function of type
 $\text{int} \rightarrow \text{int} \rightarrow \text{int}$

Currying

```
fun x y -> x + y
```

This is shorthand for:

```
fun x -> fun y -> x + y
```

This is a function of type
 $\text{int} \rightarrow \text{int} \rightarrow \text{int}$

Currying

```
fun x y -> x + y
```

This is shorthand for:

```
fun x -> (fun y -> x + y)
```

This is a function of type
`int -> (int -> int)`

Implicitly, we have
parentheses here

Partial Application

```
fun x -> fun y -> x + y
```

Partial Application

```
(fun x -> fun y -> x + y) 5
```


Partial Application

`(fun x -> fun y -> x + y) 5`



evaluates to

`fun y -> 5 + y`

Partial Application

`(fun x -> fun y -> x + y) 5`



evaluates to

`fun y -> 5 + y`

`fun x -> fun y -> x + y` has type `int -> (int -> int)`

Partial Application

`(fun x -> fun y -> x + y) 5`



evaluates to

`fun y -> 5 + y`

`fun x -> fun y -> x + y` has type `int -> (int -> int)`

`fun y -> 5 + y` has type `int -> int`

Function declarations

`let f x = e`

means

`let f = fun x -> e`

Function declarations

For example: `let f x = x + 3`

means

`let f = fun x -> x + 3`

End of recapitulation

Functions over primitive types

Consider the following functions

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

Functions over primitive types

Consider the following functions

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
add : int -> int -> int
```

```
abs : int -> int
```

```
min : int -> int -> int
```

These functions all take one or more integers, which is a standard primitive type, and return an integer

Functions over primitive types

```
add  : int -> int -> int  
abs  : int -> int  
min  : int -> int -> int
```

Moreover, these types are equivalent

```
add  : int -> (int -> int)  
abs  : int -> int  
min  : int -> (int -> int)
```


Why do we do this?

But why though?

```
let add a b = a + b (int -> (int -> int))  
let abs a = if a < 0 then -a else a (int -> int)  
let min a b = if a <= b then a else b (int -> (int -> int))
```

I am used to this and it feels simpler

```
let add (a, b) = a + b (int * int -> int)  
let abs a = if a < 0 then -a else a (int -> int)  
let min (a, b) = if a <= b then a else b (int * int -> int)
```


Short answer

The curried version

```
let add a b = a + b (int -> (int -> int))
let abs a = if a < 0 then -a else a (int -> int)
let min a b = if a <= b then a else b (int -> (int -> int))
```

Is more flexible than the uncurried one

```
let add (a, b) = a + b (int * int -> int)
let abs a = if a < 0 then -a else a (int -> int)
let min (a, b) = if a <= b then a else b (int * int -> int)
```

Short answer

This version allows you to give the arguments one by one

```
let add a b = a + b (int -> (int -> int))  
let abs a = if a < 0 then -a else a (int -> int)  
let min a b = if a <= b then a else b (int -> (int -> int))
```

This version forces you to give all arguments at once

```
let add (a, b) = a + b (int * int -> int)  
let abs a = if a < 0 then -a else a (int -> int)  
let min (a, b) = if a <= b then a else b (int * int -> int)
```

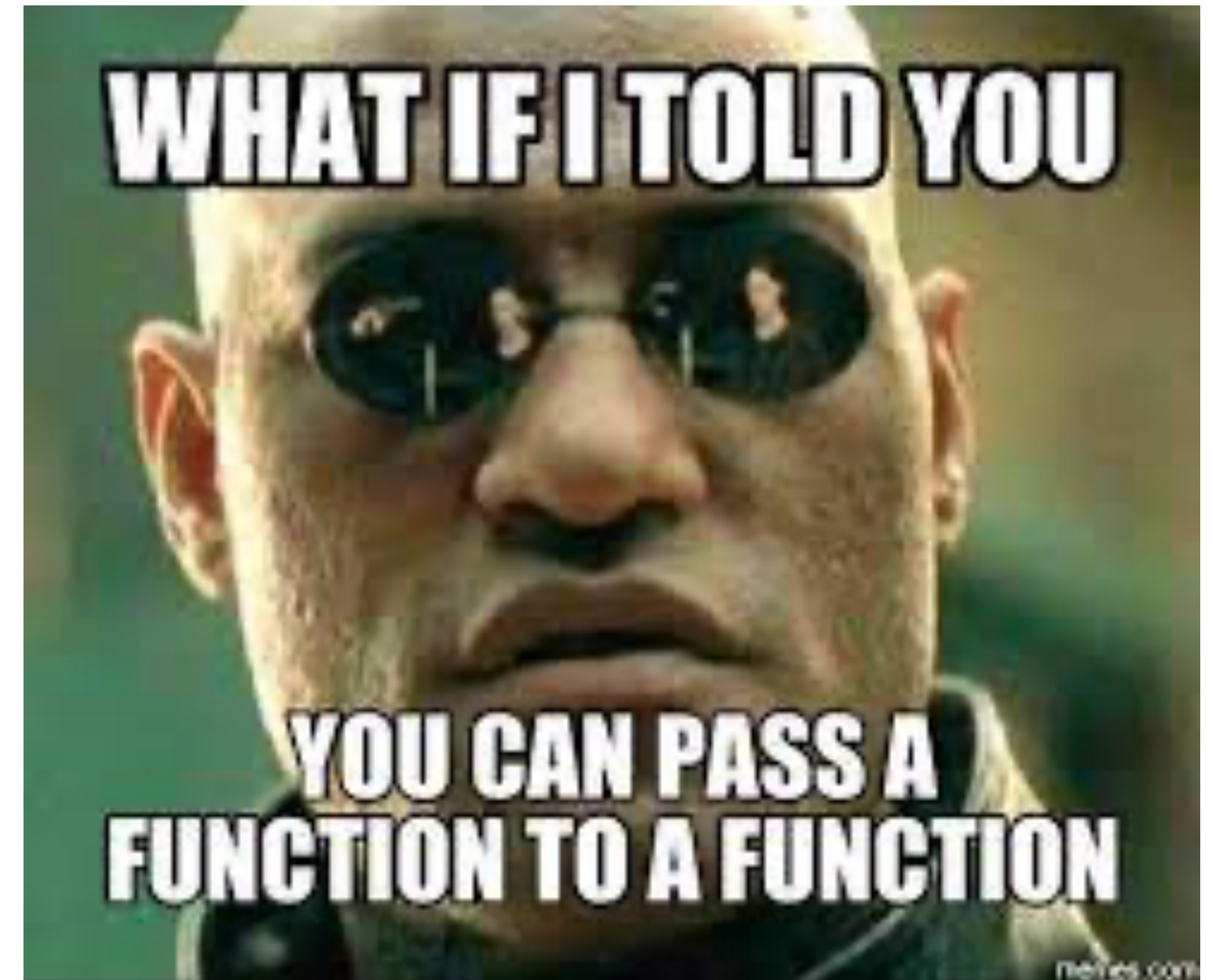
Slightly longer answer

It's a distant cousin to sub typing in object-oriented languages

- A subclass is a specialisation of a superclass
- A function that has been given some of its arguments treat those arguments as constant, in effect it is a more concrete version of the original function

Long answer

- One of the main points of this lecture
- Exceptionally useful when combined with higher-order functions



Higher-order functions

A higher-order function is any function that takes other functions as arguments

```
let add a b = a + b
let abs a   = if a < 0 then -a else a
let min a b = if a <= b then a  else b
```

First-order
functions

```
let doOne f a = f a
let doTwo f a b = f a b
```

Higher-order
functions

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3) ~> if -3 < 0 then -(-3) else -3
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)  $\leadsto$  3
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)  $\leadsto$  3
```

```
doOne abs (-3)
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)  $\leadsto$  3
```

```
doOne abs (-3)  $\leadsto$ 
```

```
(fun a -> if a < 0 then -a else a) -3
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)  $\leadsto$  3
```

```
doOne abs (-3)  $\leadsto$ 
```

```
(fun a -> if a < 0 then -a else a) -3
```

```
Remember abs = fun a -> if a < 0 then -a else a
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)  $\leadsto$  3
```

```
doOne abs (-3)  $\leadsto$ 
```

```
(fun a -> if a < 0 then -a else a) -3  $\leadsto$ 
```

```
if -3 < 0 then -(-3) else -3
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3)  $\leadsto$  if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)  $\leadsto$  3
```

```
doOne abs (-3)  $\leadsto$ 
```

```
(fun a -> if a < 0 then -a else a) -3  $\leadsto$ 
```

```
if -3 < 0 then -(-3) else -3  $\leadsto$  -(-3)
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
abs (-3) ~> if -3 < 0 then -(-3) else -3 ~> -(-3) ~> 3
```

```
doOne abs (-3) ~>
```

```
(fun a -> if a < 0 then -a else a) -3 ~>
```

```
if -3 < 0 then -(-3) else -3 ~> -(-3) ~> 3
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

Mentimeter quiz

add 5 7 $\leadsto$???

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> ???
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

```
doTwo add 5 7
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

```
doTwo add 5 7 ~> doTwo (fun a b -> a + b) 5 7
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

```
doTwo add 5 7 ~> doTwo (fun a b -> a + b) 5 7 ~>  
  (fun a b -> a + b) 5 7
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

```
doTwo add 5 7 ~> doTwo (fun a b -> a + b) 5 7 ~>  
  (fun a b -> a + b) 5 7 ~> (fun b -> 5 + b) 7
```

How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

```
doTwo add 5 7 ~> doTwo (fun a b -> a + b) 5 7 ~>
  (fun a b -> a + b) 5 7 ~> (fun b -> 5 + b) 7 ~>
  5 + 7
```


How are they used?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
add 5 7 ~> (fun b -> 5 + b) 7 ~> 5 + 7 ~> 12
```

```
doTwo add 5 7 ~> doTwo (fun a b -> a + b) 5 7 ~>  
  (fun a b -> a + b) 5 7 ~> (fun b -> 5 + b) 7 ~>  
  5 + 7 ~> 12
```

Key takeaway

`add 5 7 ~ (fun b -> 5 + b) 7 ~ 5 + 7 ~ 12`

`doTwo add 5 7 ~ doTwo (fun a b -> a + b) 5 7 ~
 (fun a b -> a + b) 5 7 ~ (fun b -> 5 + b) 7 ~
 5 + 7 ~ 12`

Passing functions as arguments to other functions is just as easy as passing arguments of any other type

What are their types?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
add : int -> (int -> int)
```

```
abs : int -> int
```

```
min : int -> (int -> int)
```

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
doOne : ('a -> 'b) -> 'a -> 'b
```

```
doTwo : ('a -> 'b -> 'c) -> 'a -> 'b -> 'c
```


What are their types?

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
add : int -> (int -> int)
```

```
abs : int -> int
```

```
min : int -> (int -> int)
```

Function arguments

```
let doOne f a = f a
```

```
let doTwo f a b = f a b
```

```
doOne : ('a -> 'b) -> 'a -> 'b
```

```
doTwo : ('a -> 'b -> 'c) -> 'a -> 'b -> 'c
```

Function types

We know from last week that

`'a -> 'b -> 'c` and `'a -> ( 'b -> 'c )`

are the same type

Function types

We know from last week that

$'a \rightarrow 'b \rightarrow 'c$ and $'a \rightarrow ('b \rightarrow 'c)$

are the same type

What about

$'a \rightarrow 'b \rightarrow 'c$ and $('a \rightarrow 'b) \rightarrow 'c$

Function types

We know from last week that

$'a \rightarrow 'b \rightarrow 'c$ and $'a \rightarrow ('b \rightarrow 'c)$

are the same type

What about

$'a \rightarrow 'b \rightarrow 'c$ and $('a \rightarrow 'b) \rightarrow 'c$

These are not the same. The latter takes function of type $'a \rightarrow 'b$ as an argument and not two arguments of type $'a$ and $'b$

Function types

There is nothing magical here, the functions with type

- `int -> int -> int` takes two integers and returns an integer
- `int -> string -> char` takes an integer and a string and returns a character
- `bool -> int` takes a boolean and returns an integer

Function types

There is nothing magical here, the functions with type

- $(\text{int} \rightarrow \text{int}) \rightarrow \text{int}$ takes a function of type $\text{int} \rightarrow \text{int}$ and returns an integer.
- $\text{int} \rightarrow (\text{string} \rightarrow \text{char}) \rightarrow \text{char}$ takes an integer and a function of type $\text{string} \rightarrow \text{char}$ and returns a character
- $(\text{int} \rightarrow \text{bool}) \rightarrow (\text{int} \rightarrow \text{bool}) \rightarrow \text{bool}$ takes two functions of the type $\text{int} \rightarrow \text{bool}$ and returns a boolean

We have seen a few before

$(+)$ = `fun` a b $\rightarrow$ a + b and has type `int  $\rightarrow$  int  $\rightarrow$  int`

We have seen a few before

$(+)$ = `fun` a b $\rightarrow$ a + b and has type `int  $\rightarrow$  int  $\rightarrow$  int`

$(+)$ is a first-order function

We have seen a few before

$(+)$ = `fun a b -> a + b` and has type `int -> int -> int`

But also

$(|>)$ = `fun a f -> f a`

and has type

`'a -> ('a -> 'b) -> 'b`

We have seen a few before

$(+)$ = `fun a b -> a + b` and has type `int -> int -> int`

But also

$(|>)$ = `fun a f -> f a` $(>>)$ = `fun g f x -> f (g x)`

and has type

`'a -> ('a -> 'b) -> 'b`

and has type

`(('a -> 'b) ->
 ('b -> 'c) ->
 'a -> 'c)`

A few examples

```
let add a b = a + b
```

```
let abs a = if a < 0 then -a else a
```

```
let min a b = if a <= b then a else b
```

```
(|>) = fun a f -> f a
```

```
(>>) = fun g f x -> f (g x)
```

```
let add4 a b c d = a + b + c + d
```

```
let add4' a b c d = a |> add b |> add c |> add d
```

```
let add4'' b c d = add b >> add c >> add d
```

These add4 functions are all the same

A few examples

It is subjective which one is the easiest to read, but I would argue `add4`.

Note how partial application is required to make `add4'` and `add4''` work.

When we come to higher-order functions on lists the benefits of piping (`|>`) and function composition (`>>`) will become more apparent.

```
let add4 a b c d = a + b + c + d
```

```
let add4' a b c d = a |> add b |> add c |> add d
```

```
let add4'' b c d = add b >> add c >> add d
```

These `add4` functions are all the same

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**
- `x :: xs` is a list with **head** `x` and **tail** `xs`

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**
- `x :: xs` is a list with **head** `x` and **tail** `xs`

In addition, `[x1; x2; ... ; xn]` is short for
`x1 :: (x2 :: ... (xn :: []))`

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**
- `x :: xs` is a list with **head** `x` and **tail** `xs`

In addition, `[x1; x2; ... ; xn]` is short for
`x1 :: (x2 :: ... (xn :: []))`

`[5; 3; -23]` is short for `5 :: (3 :: (-23 :: []))`

List Type

Lists have the type `t list`

Here, `t` is the type of the elements of the list.

E.g., `[5; 3; -23]` has type `int list`

List Type

Lists have the type `t list`

Here, `t` is the type of the elements of the list.

E.g., `[5; 3; -23]` has type `int list`

```
> 5::3::-23::[];;
```

```
val it : int list = [5; 3; -23]
```


Patterns

Consider you want to transform one list into another list

- Square each element
- Add 42 to each element
- Calculate the square root of each element
- ...

Squaring a list in Java and F#

```
public static List<Integer> squareList (List<Integer> input) {  
    List<Integer> result = new ArrayList<>();  
  
    for(int el : input) { result.add(el * el); }  
  
    return result;  
}
```

Squaring a list in Java and F#

```
public static List<Integer> squareList (List<Integer> input) {  
    List<Integer> result = new ArrayList<>();  
  
    for(int el : input) { result.add(el * el); }  
  
    return result;  
}
```

```
let rec squareList =  
    function  
    | [] -> []  
    | x::xs -> x * x :: (squareList xs)
```


Squaring a list in Java and F#

```
public static List<Integer> squareList (List<Integer> input) {  
    List<Integer> result = new ArrayList<>();  
  
    for(int el : input) { result.add(el * el); }  
  
    return result;  
}
```

```
let rec squareList =
```

```
    function
```

```
    | [] -> []
```

```
    | x::xs -> x * x :: (squareList xs)
```

```
squareList has type int list -> int list
```

Adding 42 to a list in Java and F#

```
public static List<Integer> add42List (List<Integer> input) {  
    List<Integer> result = new ArrayList<>();  
  
    for(int el : input) { result.add(el + 42); }  
  
    return result;  
}
```

```
let rec add42List =  
    function  
    | [] -> []  
    | x::xs -> x + 42 :: (add42List xs)  
  
add42List has type int list -> int list
```

Merging the functions

```
public static <T, U> List<U> map (Function<T, U> mapper,  
                                List<T> input) {  
    List<U> result = new ArrayList<>();  
  
    for(T el : input) { result.add(mapper.apply(el)); }  
  
    return result;  
}
```


Merging the functions

```
public static <T, U> List<U> map (Function<T, U> mapper,
                                   List<T> input) {
    List<U> result = new ArrayList<>();

    for(T el : input) { result.add(mapper.apply(el)); }

    return result;
}
```

```
let rec map f =
  function
```

```
| [] -> []
```

```
| x::xs -> f x :: (map f xs)
```

map has type $('a \rightarrow 'b) \rightarrow 'a \text{ list} \rightarrow 'b \text{ list}$

Using maps in Java

```
public static <T, U> List<U> map (Function<T, U> mapper,  
                                List<T> input) {  
    List<U> result = new ArrayList<>();  
    for(T el : input) { result.add(mapper.apply(el)); }  
    return result;  
}
```

```
public static int square (int x) { return x * x; }  
public static int add42  (int x) { return x + 42; }
```

```
public static List<Integer> squareList(List<Integer> input) {  
    return map(Lecture2::square, input);  
}
```

```
public static List<Integer> add42List(List<Integer> input) {  
    return map(Lecture2::add42, input);  
}
```

Using maps in F#

```
let rec map f =  
    function  
    | [] -> []  
    | x::xs -> f x :: (map f xs)  
  
let square x = x * x  
let add42 x = x + 42  
  
let squareList lst = map square lst  
let add42List lst = map add42 lst
```


Using maps in F#

```
let rec map f =  
    function  
    | [] -> []  
    | x::xs -> f x :: (map f xs)  
  
let square x = x * x  
let add42 x = x + 42  
  
let squareList lst = map square lst  
let add42List lst = map add42 lst  
  
    squareList [1; 2; 3; 4; 5] = [1; 4; 9; 16; 25]  
    add42List [1; 2; 3; 4; 5] = [43; 44; 45; 46; 47]
```

Streams in Java

Java has supports for streams which is the preferred way of working with higher-order functions in Java

```
public static int square (int x) { return x * x; }
public static int add42. (int x) { return x + 42; }

public static List<Integer> squareList(List<Integer> input) {
    return input.stream().map(Lecture2::square).toList();
}

public static List<Integer> add42List(List<Integer> input) {
    return input.stream().map(Lecture2::add42).toList();
}
```

Maps in the list library

`List.map` is a library function

```
let rec map f =
  function
  | []          -> []
  | x :: xs     -> f x :: map f xs
```

`map : ('a -> 'b) -> 'a list -> 'b list`

`List.map f [x0; x1; ... ; xn-1] = [f x0; f x1; ... ; f xn-1]`

Maps (evaluation)

```

let rec map f =
  function
  | []          -> []
  | x :: xs    -> f x :: map f xs
addElems [(1, 2); (3, 4); (5, 6)]

let add (a, b) = a + b
let addElems lst = map add lst

```

Maps (evaluation)

```
let rec map f =
```

```
  function
```

```
    | []          -> []
```

```
    | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```

Maps (evaluation)

```
let rec map f =  
  function
```

```
    | []          -> []
```

```
    | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```


Maps (evaluation)

```
let rec map f =  
  function
```

```
  | []          -> []
```

```
  | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)] ~
```

```
1 + 2 :: map add [(3, 4); (5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```

Maps (evaluation)

```
let rec map f =  
  function
```

```
    | []          -> []
```

```
    | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)] ~
```

```
1 + 2 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: map add [(3, 4); (5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```

Maps (evaluation)

```
let rec map f =  
  function
```

```
    | []          -> []
```

```
    | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)] ~
```

```
1 + 2 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: add (3, 4) :: map add [(5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```


Maps (evaluation)

```
let rec map f =  
  function
```

```
  | []          -> []
```

```
  | x :: xs    -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)] ~
```

```
1 + 2 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: add (3, 4) :: map add [(5, 6)] ~
```

```
3 :: 3 + 4 :: map add [(5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```

Maps (evaluation)

```
let rec map f =  
  function
```

```
  | []          -> []
```

```
  | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)] ~
```

```
1 + 2 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: add (3, 4) :: map add [(5, 6)] ~
```

```
3 :: 3 + 4 :: map add [(5, 6)] ~ 3 :: 7 :: map add [(5, 6)]
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```

Maps (evaluation)

```

let rec map f =
  function
  | []          -> []
  | x :: xs    -> f x :: map f xs
addElems [(1, 2); (3, 4); (5, 6)] ~
map add [(1, 2); (3, 4); (5, 6)] ~
add (1, 2) :: map add [(3, 4); (5, 6)] ~
1 + 2 :: map add [(3, 4); (5, 6)] ~
3 :: map add [(3, 4); (5, 6)] ~
3 :: add (3, 4) :: map add [(5, 6)] ~
3 :: 3 + 4 :: map add [(5, 6)] ~ 3 :: 7 :: map add [(5, 6)] ~
3 :: 7 :: add (5, 6) :: map add []
  
```

`let add (a, b) = a + b`
`let addElems lst = map add lst`

Maps (evaluation)

```
let rec map f =  
  function
```

```
  | []          -> []
```

```
  | x :: xs     -> f x :: map f xs
```

```
addElems [(1, 2); (3, 4); (5, 6)] ~
```

```
map add [(1, 2); (3, 4); (5, 6)] ~
```

```
add (1, 2) :: map add [(3, 4); (5, 6)] ~
```

```
1 + 2 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: map add [(3, 4); (5, 6)] ~
```

```
3 :: add (3, 4) :: map add [(5, 6)] ~
```

```
3 :: 3 + 4 :: map add [(5, 6)] ~ 3 :: 7 :: map add [(5, 6)] ~
```

```
3 :: 7 :: add (5, 6) :: map add [] ~
```

```
3 :: 7 :: 5 + 6 :: map add []
```

```
let add (a, b) = a + b
```

```
let addElems lst = map add lst
```

Maps (evaluation)

```

let rec map f =
  function
  | []          -> []
  | x :: xs    -> f x :: map f xs
addElems [(1, 2); (3, 4); (5, 6)] ~
map add [(1, 2); (3, 4); (5, 6)] ~
add (1, 2) :: map add [(3, 4); (5, 6)] ~
1 + 2 :: map add [(3, 4); (5, 6)] ~
3 :: map add [(3, 4); (5, 6)] ~
3 :: add (3, 4) :: map add [(5, 6)] ~
3 :: 3 + 4 :: map add [(5, 6)] ~ 3 :: 7 :: map add [(5, 6)] ~
3 :: 7 :: add (5, 6) :: map add [] ~
3 :: 7 :: 5 + 6 :: map add [] ~ 3 :: 7 :: 11 :: map add []

```

`let add (a, b) = a + b`
`let addElems lst = map add lst`

Maps (evaluation)

```

let rec map f =
  function
  | []          -> []
  | x :: xs    -> f x :: map f xs
addElems [(1, 2); (3, 4); (5, 6)] ~
map add [(1, 2); (3, 4); (5, 6)] ~
add (1, 2) :: map add [(3, 4); (5, 6)] ~
1 + 2 :: map add [(3, 4); (5, 6)] ~
3 :: map add [(3, 4); (5, 6)] ~
3 :: add (3, 4) :: map add [(5, 6)] ~
3 :: 3 + 4 :: map add [(5, 6)] ~ 3 :: 7 :: map add [(5, 6)] ~
3 :: 7 :: add (5, 6) :: map add [] ~
3 :: 7 :: 5 + 6 :: map add [] ~ 3 :: 7 :: 11 :: map add [] ~
3 :: 7 :: 11 :: []

```

```

let add (a, b) = a + b
let addElems lst = map add lst

```


Maps (evaluation)

```

let rec map f =
  function
  | []          -> []
  | x :: xs    -> f x :: map f xs
addElems [(1, 2); (3, 4); (5, 6)] ~
map add [(1, 2); (3, 4); (5, 6)] ~
add (1, 2) :: map add [(3, 4); (5, 6)] ~
1 + 2 :: map add [(3, 4); (5, 6)] ~
3 :: map add [(3, 4); (5, 6)] ~
3 :: add (3, 4) :: map add [(5, 6)] ~
3 :: 3 + 4 :: map add [(5, 6)] ~ 3 :: 7 :: map add [(5, 6)] ~
3 :: 7 :: add (5, 6) :: map add [] ~
3 :: 7 :: 5 + 6 :: map add [] ~ 3 :: 7 :: 11 :: map add [] ~
3 :: 7 :: 11 :: [] ~ [3; 7; 11]

```

```

let add (a, b) = a + b
let addElems lst = map add lst

```

Filters

Patterns

Consider you want to remove elements from a list

- Remove all even numbers
- Remove all odd numbers
- Remove all vehicles with two wheels
- ...

Remove odd numbers in Java and F#

```
public static List<Integer> keepEven (List<Integer> input) {  
    List<Integer> result = new ArrayList<>();  
  
    for(int el : input) { if (el % 2 == 0) { result.add(el); } }  
  
    return result;  
}
```

Remove odd numbers in Java and F#

```
public static List<Integer> keepEven (List<Integer> input) {
    List<Integer> result = new ArrayList<>();

    for(int el : input) { if (el % 2 == 0) { result.add(el); } }

    return result;
}
```

```
let rec keepEven =
    function
    | [] -> []
    | x::xs when x % 2 = 0 -> x :: (keepEven xs)
    | x::xs -> keepEven xs
```

Remove odd numbers in Java and F#

```
public static List<Integer> keepEven (List<Integer> input) {
    List<Integer> result = new ArrayList<>();

    for(int el : input) { if (el % 2 == 0) { result.add(el); } }

    return result;
}
```

```
let rec keepEven =
    function
    | []                -> []
    | x::xs when x % 2 = 0 -> x :: (keepEven xs)
    | x::xs             -> keepEven xs
```

keepEven has type `int list -> int list`

Remove even numbers in Java and F#

```
public static List<Integer> keepOdd (List<Integer> input) {
    List<Integer> result = new ArrayList<>();

    for(int el : input) { if (el % 2 == 1) { result.add(el); } }

    return result;
}
```

```
let rec keepOdd =
    function
    | []                -> []
    | x::xs when x % 2 = 1 -> x :: (keepOdd xs)
    | x::xs             -> keepOdd xs

keepOdd has type int list -> int list
```

The filter function

```
let rec keepOdd =
  function
  | []                -> []
  | x::xs when x % 2 = 1 -> x :: (keepOdd xs)
  | x::xs             -> keepOdd xs
```

This function can be generalised

```
let rec filter predicate =
  function
  | []                -> []
  | x::xs when predicate x -> x :: (filter xs)
  | x::xs             -> filter xs
```

filter has type ('a -> bool) -> 'a list -> 'a list

The filter function

```
let rec filter predicate =
  function
  | []                -> []
  | x::xs when predicate x -> x :: (filter xs)
  | x::xs             -> filter xs
```

filter has type $('a \rightarrow \text{bool}) \rightarrow 'a \text{ list} \rightarrow 'a \text{ list}$

```
keepEven lst = filter (fun x -> x % 2 = 0) lst
keepOdd  lst = filter (fun x -> x % 2 = 1) lst
```


The filter function

```
let rec filter predicate =
  function
  | []                -> []
  | x::xs when predicate x -> x :: (keepOdd xs)
  | x::xs             -> keepOdd xs
```

filter has type $('a \rightarrow \text{bool}) \rightarrow 'a \text{ list} \rightarrow 'a \text{ list}$

```
keepEven lst = filter (fun x -> x % 2 = 0) lst
```

```
keepOdd  lst = filter (fun x -> x % 2 = 1) lst
```

```
keepEven [1; 2; 3; 4; 5] = [2; 4]
```

```
keepOdd  [1; 2; 3; 4; 5] = [1; 3; 5]
```

Filters in the list library

`List.filter` is a library function

```
let rec filter predicate =  
  function  
  | []                -> []  
  | x::xs when predicate x -> x :: (keepOdd xs)  
  | x::xs             -> keepOdd xs
```

`filter` has type $('a \rightarrow \text{bool}) \rightarrow 'a \text{ list} \rightarrow 'a \text{ list}$

`filter pred [x1; x2; ...; xn]`

returns a list of all elements x_i such that `pred xi = true`

Filters (evaluation)

```
let rec filter predicate =  
  function  
  | []                -> []  
  | x :: xs when predicate x -> x :: (filter predicate xs)  
  | _ :: xs           -> filter predicate xs  
  
let removeNegative = filter ((<=) 0)
```

What is ((<=) 0)?

Filters (evaluation)

```
let rec filter predicate =  
  function  
  | []                -> []  
  | x :: xs when predicate x -> x :: (filter predicate xs)  
  | _ :: xs           -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
filter ((<=) 0) = filter (fun x -> 0 <= x)
```

Filters (evaluation)

```
let rec filter predicate =  
  function  
  | []                -> []  
  | x :: xs when predicate x -> x :: (filter predicate xs)  
  | _ :: xs           -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0]
```

Filters (evaluation)

```
let rec filter predicate =  
  function  
  | []                -> []  
  | x :: xs when predicate x -> x :: (filter predicate xs)  
  | _ :: xs           -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0]
```


Filters (evaluation)

```
let rec filter predicate =
  function
  | []                -> []
  | x :: xs when predicate x -> x :: (filter predicate xs)
  | _ :: xs           -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0] ~>
4 :: filter ((<=) 0) [-5; 6; 0]
```

Filters (evaluation)

```
let rec filter predicate =
  function
  | []                -> []
  | x :: xs when predicate x -> x :: (filter predicate xs)
  | _ :: xs          -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0] ~>
4 :: filter ((<=) 0) [-5; 6; 0] ~> 4 :: filter ((<=) 0) [6; 0]
```

Filters (evaluation)

```
let rec filter predicate =
  function
  | []                -> []
  | x :: xs when predicate x -> x :: (filter predicate xs)
  | _ :: xs          -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0] ~>
4 :: filter ((<=) 0) [-5; 6; 0] ~> 4 :: filter ((<=) 0) [6; 0] ~>
4 :: 6 :: filter ((<=) 0) [0]
```


Filters (evaluation)

```
let rec filter predicate =
  function
  | []                -> []
  | x :: xs when predicate x -> x :: (filter predicate xs)
  | _ :: xs          -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0] ~>
4 :: filter ((<=) 0) [-5; 6; 0] ~> 4 :: filter ((<=) 0) [6; 0] ~>
4 :: 6 :: filter ((<=) 0) [0] ~>
4 :: 6 :: 0 :: filter ((<=) 0) []
```

Filters (evaluation)

```
let rec filter predicate =
  function
  | []                -> []
  | x :: xs when predicate x -> x :: (filter predicate xs)
  | _ :: xs          -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0] ~>
4 :: filter ((<=) 0) [-5; 6; 0] ~> 4 :: filter ((<=) 0) [6; 0] ~>
4 :: 6 :: filter ((<=) 0) [0] ~>
4 :: 6 :: 0 :: filter ((<=) 0) [] ~> 4 :: 6 :: 0 :: []
```

Filters (evaluation)

```
let rec filter predicate =
  function
  | []                -> []
  | x :: xs when predicate x -> x :: (filter predicate xs)
  | _ :: xs          -> filter predicate xs
```

```
let removeNegative = filter ((<=) 0)
```

```
removeNegative [4; -5; 6; 0] ~> filter ((<=) 0) [4; -5; 6; 0] ~>
4 :: filter ((<=) 0) [-5; 6; 0] ~> 4 :: filter ((<=) 0) [6; 0] ~>
4 :: 6 :: filter ((<=) 0) [0] ~>
4 :: 6 :: 0 :: filter ((<=) 0) [] ~> 4 :: 6 :: 0 :: [] ~>
[4; 6; 0]
```


Folds

Before we start, folds are what comes the closest to the standard loops that you are used to, even though this is not always directly apparent. They are exceptionally useful.

Folds (loops)

The following Java program sums all elements of a list

```
public static int sumList(List<Integer> input) {  
    int accumulator = 0;  
    for(int el : input) { accumulator += el; }  
    return accumulator;  
}
```


Folds (loops)

The following Java program sums all elements of a list

```
public static int sumList(List<Integer> input) {  
    int accumulator = 0;  
    for(int el : input) { accumulator += el; }  
    return accumulator;  
}
```

and the following does the same in F#

```
let rec sumList =  
    function  
    | []      -> 0  
    | x::xs  -> x + sumList xs
```

Folds (loops)

The following Java program sums all elements of a list

```
public static int sumList(List<Integer> input) {
    int accumulator = 0;
    for(int el : input) { accumulator += el; }
    return accumulator;
}
```

Another version that follows the loop a bit more closely

```
let sumList input =
  let rec loop accumulator =
    function
      | []      -> accumulator
      | el::xs  -> loop (accumulator + el) xs
  loop 0 input
```

Folds (loops)

```
let sumList input =  
  let rec loop accumulator =  
    function  
      | []      -> accumulator  
      | e1::xs  -> loop (accumulator + e1) xs  
  loop 0 input
```

More generally

```
let rec fold folder accumulator =  
  function  
    | []      -> accumulator  
    | e1::xs  -> fold folder (folder accumulator e1) xs
```


Folds (loops)

```
let sumList input =  
  let rec loop accumulator =  
    function  
      | []      -> accumulator  
      | e1::xs  -> loop (accumulator + e1) xs  
  loop 0 input
```

More generally

```
let rec fold folder accumulator =  
  function  
    | []      -> accumulator  
    | e1::xs  -> fold folder (folder accumulator e1) xs  
  
sumList lst = fold (fun a b -> a + b) 0 lst
```

Folds (loops)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | e1::xs  -> fold folder (folder accumulator e1) xs
```

```
sumList lst = fold (fun a b -> a + b) 0 lst
```

$\text{sumList } [1;2;3;4;5] = (((((0 + 1) + 2) + 3) + 4) + 5) = 15$

Folds (loops)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | el::xs  -> fold folder (folder accumulator el) xs
```

```
fold : ('b -> 'a -> 'b) -> 'b -> 'a list -> 'b
```


Folds (loops)

```
let rec fold folder accumulator =
  function
  | []      -> accumulator
  | e1::xs  -> fold folder (folder accumulator e1) xs
```

`fold : ('b -> 'a -> 'b) -> 'b -> 'a list -> 'b`

Mathematically

$$\text{fold } f \text{ acc } [x_1; x_2; \dots; x_n]$$

returns

$$f \text{ } (\dots (f \text{ } (f \text{ acc } x_1) \text{ } x_2) \dots x_{n-1}) \text{ } x_n$$

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs  
  
fold (-) 0 [1; 2; 3]
```

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs  
  
fold (-) 0 [1; 2; 3] ~  
fold (-) ((-) 0 1) [2; 3]
```


Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]`

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]`

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]  $\leadsto$`

`fold (-) ((-) -1 2) [3]`

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]  $\leadsto$`

`fold (-) ((-) -1 2) [3]  $\leadsto$`

`fold (-) (-1 - 2) [3]`

Folds (evaluation)

```
let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]  $\leadsto$`

`fold (-) ((-) -1 2) [3]  $\leadsto$`

`fold (-) (-1 - 2) [3]  $\leadsto$  fold (-) -3 [3]`

Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) []

```


Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) [] ~>
fold (-) (-3 - 3) []

```

Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) [] ~>
fold (-) (-3 - 3) [] ~> fold (-) -6 []

```

Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) [] ~>
fold (-) (-3 - 3) [] ~> fold (-) -6 [] ~>
-6

```


Quote from student towards the end of the course
(paraphrased)

"I just realised that with folds you can do
absolutely anything!"

Quote from student towards the end of the course
(paraphrased)

"I just realised that with folds you can do
absolutely anything!"

Not quite true, but the TAs were dancing

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers
- Obtain the last digit from each number
- Construct a string containing the last digit of each number

What higher-order functions should we use for these steps?

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

```
mergeLastDigits [123; 124; 5; 1233] = [123; 5; 1233]
```

```
let mergeLastDigits lst =
```

```
(filter (fun x -> x % 2 = 1) lst)
```


Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

```
mergeLastDigits [123; 124; 5; 1233] = [3; 5; 3]
```

```
let mergeLastDigits lst =
```

```
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

```
mergeLastDigits [123; 124; 5; 1233] = "353"  
let mergeLastDigits lst =  
  fold (fun acc x -> acc + string x) ""  
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

Example

```
let mergeLastDigits lst =  
  fold (fun acc x -> acc + string x) ""  
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

which is the same as

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```


Example

```
let mergeLastDigits lst =  
  fold (fun acc x -> acc + string x) ""  
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

which is the same as

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```

This is MUCH easier to read

Example

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```

which is the same as

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> acc + string x) ""
```


Example

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> acc + string x) ""
```

which is the same as

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> x |> string |> (+) acc) ""
```


Example

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> x |> string |> (+) acc) ""
```

which is the same as

Which is not easier to read

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc -> string >> (+) acc) ""
```

And the winner is....

In my highly subjective opinion... one of these two

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> acc + string x) ""
```

Things to practice on

Become good friends with

- Piping (`|>`)
- Function composition (`>>`)
- `map`
- `filter`
- `fold`

Questions?

Appendix (More material)

Records

Records allow us to structure data

```
type person =  
  { firstname : string;  
    lastname  : string;  
    age       : int }
```

Records can be of arbitrary size

Records

Records are declared by providing input to
all of their fields
(no partial application)

```
let Jesper =  
  { firstname = "Jesper";  
    lastname = "Bengtson";  
    age = 41 } ;;  
val Jesper : person =  
  { firstname = "Jesper";  
    lastname = "Bengtson";  
    age = 41 ; }
```

Records

Records are declared by providing input to
all of their fields
(no partial application)

All field names must match with an
already declared type

```
val person = {  
  firstname = "Jesper";  
  lastname = "Bengtson";  
  age = 41;  
}
```

Records

Individual fields are accessed using their name

```
jesper.firstname;;  
val it : string = "Jesper"
```

```
jesper.lastname;;  
val it : string = "Bengtson"
```

```
jesper.age;;  
val it : int = 41
```


let-patterns

It is possible to obtain values from compound statements

```
let { firstname = fn;  
    lastname = ln;  
    age = x } = jesper in  
    (fn, ln, x);;  
val it : string * string * int =  
    ("Jesper", "Bengtson", 41)
```

let-patterns

It is possible to obtain values from compound statements

```
let (a, b) = (5, true) in (b, a);;  
val it : bool * int = (true, 5)
```


Exists

Patterns

Consider the following function

```
let rec containsPositive =  
  function  
  | []                -> false  
  | x :: _ when x > 0 -> true  
  | _ :: xs          -> containsPositive xs
```

`containsPositive : int list -> bool`

`containsPositive [-5; 4; 6; 0] = true`

`containsPositive [-5; -4; -6; 0] = false`

Exists

For a predicate $p : 'a \rightarrow \text{bool}$,

$\text{exists } p [x_1; x_2; \dots; x_n]$

returns true if there is an element x_i such that $p x_i$ holds, and false otherwise

Exists with loops

`exists p [x1; x2; ...; xn]`

returns true if there is an element x_i such that $p\ x_i$ holds, and false otherwise

`let b = exists p xs` vs.

```
b = false;
for (i = 0; i < xs.Length; i++) {
    b = b || p (xs[i]);
}
```


Exists

```
let rec exists p =  
  function  
  | []                -> false  
  | x :: _ when p x  -> true  
  | _ :: xs          -> exists p xs;;  
  
exists : ('a -> bool) -> 'a list -> bool
```

Exists

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs;;  
  
exists : ('a -> bool) -> 'a list -> bool  
  
let containsPositive = exists ((<) 0)  
  
containsPositive [-5; 4; 6; 0] = true  
  
containsPositive [-5; -4; -6; 0] = false
```

Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs  
  
let containsPositive = exists ((<) 0)  
  
containsPositive [-5; 4; 6; 0]
```


Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; 4; 6; 0] ~  
exists ((<) 0) [-5; 4; 6; 0]
```

Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; 4; 6; 0] ~  
exists ((<) 0) [-5; 4; 6; 0] ~  
exists [4; 6; 0]
```

Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; 4; 6; 0] ~  
exists ((<) 0) [-5; 4; 6; 0] ~  
exists [4; 6; 0] ~ true
```


Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs  
  
let containsPositive = exists ((<) 0)  
  
containsPositive [-5; -4; -6; 0]
```

Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs  
  
let containsPositive = exists ((<) 0)  
  
containsPositive [-5; -4; -6; 0] ~  
exists ((<) 0) [-5; -4; -6; 0]
```

Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs  
  
let containsPositive = exists ((<) 0)  
  
containsPositive [-5; -4; -6; 0] ~  
exists ((<) 0) [-5; -4; -6; 0] ~  
exists ((<) 0) [-4; -6; 0]
```


Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; -4; -6; 0] ~>
```

```
exists ((<) 0) [-5; -4; -6; 0] ~>
```

```
exists ((<) 0) [-4; -6; 0] ~>
```

```
exists ((<) 0) [-6; 0]
```

Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; -4; -6; 0] ~  
exists ((<) 0) [-5; -4; -6; 0] ~  
exists ((<) 0) [-4; -6; 0] ~  
exists ((<) 0) [-6; 0] ~  
exists ((<) 0) [0]
```

Exists (evaluation)

```
let rec exists p =
  function
  | []          -> false
  | x :: _ when p x -> true
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; -4; -6; 0] ~>
```

```
exists ((<) 0) [-5; -4; -6; 0] ~>
```

```
exists ((<) 0) [-4; -6; 0] ~>
```

```
exists ((<) 0) [-6; 0] ~>
```

```
exists ((<) 0) [0] ~>
```

```
exists ((<) 0) []
```


Exists (evaluation)

```
let rec exists p =  
  function  
  | []          -> false  
  | x :: _ when p x -> true  
  | _ :: xs     -> exists p xs
```

```
let containsPositive = exists ((<) 0)
```

```
containsPositive [-5; -4; -6; 0] ~
```

```
exists ((<) 0) [-5; -4; -6; 0] ~
```

```
exists ((<) 0) [-4; -6; 0] ~
```

```
exists ((<) 0) [-6; 0] ~
```

```
exists ((<) 0) [0] ~
```

```
exists ((<) 0) [] ~ false
```

Forall

Patterns

Consider the following function

```
let rec allPositive =  
  function  
  | []                -> true  
  | x :: xs when x <= 0 -> false  
  | x :: xs           -> containsPositive xs
```

`allPositive : int list -> bool`

`allPositive [5; -4; 6; 0] = false`

`allPositive [5; 4; 6; 1] = true`

Forall

For a predicate $p : 'a \rightarrow \text{bool}$,

`forall p [x1; x2; ...; xn]`

returns true if $p\ x_i$ holds for all elements x_i ,
and false otherwise

Forall with loops

`forall p [x1; x2; ...; xn]`

returns true if `p xi` holds for all elements `xi`,
and false otherwise

`let b = forall p xs` vs.

```
b = true;
for(i = 0; i < xs.Length; i++) {
    b = b && p (xs[i]);
}
```

Forall

```
let rec forall p =  
  function
```

```
    | []                -> true  
    | x :: xs when p x -> forall p xs;;  
    | _                -> false
```

```
forall : ('a -> bool) -> 'a list -> bool
```


Forall

```
let rec forall p =  
  function
```

```
  | []                -> true  
  | x :: xs when p x -> forall p xs;;  
  | _                -> false
```

```
forall : ('a -> bool) -> 'a list -> bool
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; -4; 6; 0] = false
```

```
allPositive [5; 4; 6; 1] = true
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []          -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs      -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; -4; 6; 0]
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []          -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs      -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; -4; 6; 0] ~  
forall ((<) 0) [5; -4; 6; 0]
```


Forall (evaluation)

```
let rec forall p =  
  function  
  | []                -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs           -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; -4; 6; 0] ~  
forall ((<) 0) [5; -4; 6; 0] ~  
forall [-4; 6; 0]
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []          -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs      -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; -4; 6; 0] ~  
forall ((<) 0) [5; -4; 6; 0] ~  
forall [-4; 6; 0] ~ false
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []          -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs      -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1]
```


Forall (evaluation)

```
let rec forall p =  
  function  
  | []                -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs           -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1] ~  
forall ((<) 0) [5; 4; 6; 1]
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []                -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs           -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1] ~  
forall ((<) 0) [5; 4; 6; 1] ~  
forall ((<) 0) [4; 6; 1]
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []                -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs           -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1] ~  
forall ((<) 0) [5; 4; 6; 1] ~  
forall ((<) 0) [4; 6; 1] ~  
forall ((<) 0) [6; 1]
```


Forall (evaluation)

```
let rec forall p =  
  function  
  | []                -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs           -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1] ~  
forall ((<) 0) [5; 4; 6; 1] ~  
forall ((<) 0) [4; 6; 1] ~  
forall ((<) 0) [6; 1] ~  
forall ((<) 0) [1]
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []          -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs      -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1] ~  
forall ((<) 0) [5; 4; 6; 1] ~  
forall ((<) 0) [4; 6; 1] ~  
forall ((<) 0) [6; 1] ~  
forall ((<) 0) [1] ~  
forall ((<) 0) []
```

Forall (evaluation)

```
let rec forall p =  
  function  
  | []                -> true  
  | x :: xs when p x -> forall p xs  
  | x :: xs           -> false
```

```
let allPositive = forall ((<) 0)
```

```
allPositive [5; 4; 6; 1] ~  
forall ((<) 0) [5; 4; 6; 1] ~  
forall ((<) 0) [4; 6; 1] ~  
forall ((<) 0) [6; 1] ~  
forall ((<) 0) [1] ~  
forall ((<) 0) [] ~ true
```


Folds (loops)

For a function f , and an initial value acc ,

`foldBack f [x1; x2; ...; xn] acc`

returns

`f x1 (f x2 (... f xn-1 (f xn acc)...))`

This particular fold is called a `foldBack` in F# because it starts at the end of the list

Folds (loops)

`foldBack f [x1; x2; ...; xn] acc`

`returns`

`f x1 (f x2 (... f xn-1 (f xn acc)...))`

`let acc = foldBack f xs init`

`VS.`

`acc = init;`

`for(i = xs.Length - 1; i >= 0; i--)`

`{`

`acc = f (xs[i], acc);`

`}`

Folds (loops)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
```


Folds (loops)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

`foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`

`foldBack (-) [1; 2; 3] 0 = 1-(2-(3-0)) = 2`

FoldBack (evaluation)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~  
(-) 1 (foldBack (-) [2; 3] 0)
```


FoldBack (evaluation)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~  
(-) 1 (foldBack (-) [2; 3] 0) ~  
(-) 1 ((-) 2 (foldBack (-) [3] 0))
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~  
(-) 1 (foldBack (-) [2; 3] 0) ~  
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~  
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0)))
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =  
  match xs with  
  | []          -> acc  
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~  
(-) 1 (foldBack (-) [2; 3] 0) ~  
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~  
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~  
(-) 1 ((-) 2 ((-) 3 0))
```


FoldBack (evaluation)

```
let rec foldBack f xs acc =
  match xs with
  | []          -> acc
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~
(-) 1 (foldBack (-) [2; 3] 0) ~
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~
(-) 1 ((-) 2 ((-) 3 0)) ~ (-) 1 ((-) 2 (3 - 0))
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =
  match xs with
  | []          -> acc
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~
(-) 1 (foldBack (-) [2; 3] 0) ~
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~
(-) 1 ((-) 2 ((-) 3 0)) ~ (-) 1 ((-) 2 (3 - 0)) ~
(-) 1 ((-) 2 3)
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =
  match xs with
  | []          -> acc
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~
(-) 1 (foldBack (-) [2; 3] 0) ~
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~
(-) 1 ((-) 2 ((-) 3 0)) ~ (-) 1 ((-) 2 (3 - 0)) ~
(-) 1 ((-) 2 3) ~ (-) 1 (2 - 3)
```


FoldBack (evaluation)

```
let rec foldBack f xs acc =
  match xs with
  | []          -> acc
  | x :: xs    -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~
(-) 1 (foldBack (-) [2; 3] 0) ~
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~
(-) 1 ((-) 2 ((-) 3 0)) ~ (-) 1 ((-) 2 (3 - 0)) ~
(-) 1 ((-) 2 3) ~ (-) 1 (2 - 3) ~ (-) 1 -1
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =
  match xs with
  | []          -> acc
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~
(-) 1 (foldBack (-) [2; 3] 0) ~
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~
(-) 1 ((-) 2 ((-) 3 0)) ~ (-) 1 ((-) 2 (3 - 0)) ~
(-) 1 ((-) 2 3) ~ (-) 1 (2 - 3) ~ (-) 1 -1 ~ 1 - -1
```

FoldBack (evaluation)

```
let rec foldBack f xs acc =
  match xs with
  | []          -> acc
  | x :: xs     -> f x (foldBack f xs acc)
```

```
foldBack (-) [1; 2; 3] 0 ~
(-) 1 (foldBack (-) [2; 3] 0) ~
(-) 1 ((-) 2 (foldBack (-) [3] 0)) ~
(-) 1 ((-) 2 ((-) 3 (foldBack (-) [] 0))) ~
(-) 1 ((-) 2 ((-) 3 0)) ~ (-) 1 ((-) 2 (3 - 0)) ~
(-) 1 ((-) 2 3) ~ (-) 1 (2 - 3) ~ (-) 1 -1 ~ 1 - -1 ~ 2
```


With folds you can do almost anything

```
map f lst ==  
  foldBack (fun x acc -> f x :: acc) lst []  
filter p lst ==  
  foldBack (fun x acc -> if p x then x :: acc else acc)  
    lst []  
  
exists p lst == fold (fun acc x -> p x || acc) false lst  
forall p lst == fold (fun acc x -> p x && acc) true ls
```

With folds you can do almost anything

```
map f lst ==  
  foldBack (fun x acc -> f x :: acc) lst []  
filter p lst ==  
  foldBack (fun x acc -> if p x then x :: acc else acc)  
  lst []  
  
exists p lst == fold (fun acc x -> p x || acc) false lst  
forall p lst == fold (fun acc x -> p x && acc) true ls  
  
exists p lst == foldBack (fun x acc -> p x || acc) lst false  
forall p lst == foldBack (fun x acc -> p x && acc) lst true
```

With folds you can do almost anything

```
map f lst ==
  foldBack (fun x acc -> f x :: acc) lst []
filter p lst ==
  foldBack (fun x acc -> if p x then x :: acc else acc)
  lst []

exists p lst == fold (fun acc x -> p x || acc) false lst
forall p lst == fold (fun acc x -> p x && acc) true ls

exists p lst == foldBack (fun x acc -> p x || acc) lst false
forall p lst == foldBack (fun x acc -> p x && acc) lst true

exists p == foldBack (p >> (||)) false
forall p == foldBack (p >> (&&)) true
```


With folds you can do almost anything

This last one may seem magical

```
forall p lst == foldBack (fun x acc -> p x && acc) lst true
```

```
forall p == foldBack (p >> (&&)) true
```

With folds you can do almost anything

This last one may seem magical

```
p >> ( && )
```

```
forall p lst == foldBack (fun x acc -> p x && acc) lst true
```

```
forall p == foldBack (p >> ( && )) true
```

With folds you can do almost anything

This last one may seem magical

```
f >> g = fun x -> g (f x)
```

```
p >> (&&) ==  
fun x => (&&) (p x)
```

```
forall p lst == foldBack (fun x acc -> p x && acc) lst true
```

```
forall p == foldBack (p >> (&&)) true
```


With folds you can do almost anything

This last one may seem magical

```
p >> ( && ) ==  
fun x => ( && ) (p x) ==  
fun x acc => ( && ) (p x) acc
```

```
forall p lst == foldBack (fun x acc -> p x && acc) lst true
```

```
forall p == foldBack (p >> ( && )) true
```

With folds you can do almost anything

This last one may seem magical

```
p >> ( && ) ==  
fun x => ( && ) (p x) ==  
fun x acc => ( && ) (p x) acc ==  
fun x acc => (p x) && acc
```

```
forall p lst == foldBack (fun x acc -> p x && acc) lst true
```

```
forall p == foldBack (p >> ( && )) true
```